

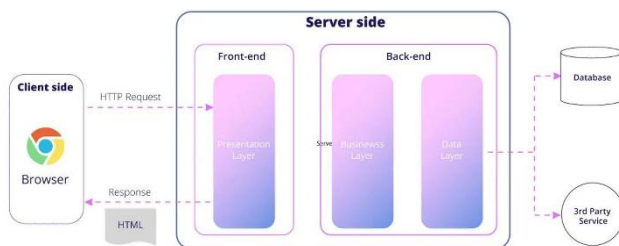
Table des matières

Définition.....	2
Le MVC avec Angular :	3
Installations : commandes importantes	3
Structure d'un projet Angular.....	4
TypeScript	4
DataType en TypeScript.....	6
Fonction et classes en TypeScript.....	8
Bonus.....	9
Composants Angular	10
Décorateur	10
Sélecteur	13
Bootstrap.....	13
Directives	15
Structural Directive	15
Attribut Directives	16
Tubes / Pipes.....	18
Service	19
Event Binding.....	21
Two Way Binding / liasion bidirectionelle	22
Routage	23
Authgard.....	25
Formulaire / form	27
Design Pattern	34
Backend : Json Server	36
http service.....	38
Laravel	40

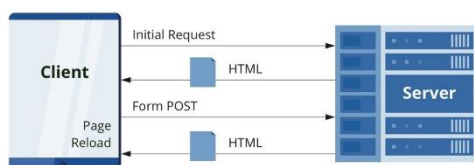
Définition

Site Web	Groupe de pages statiques accessibles via un domaine.
Application Web	Logiciel accessible via navigateur utilisant HTML/CSS/JS.
3-tier architecture	Présentation (Frontend), Application (Logique), Données (Backend)
DOM (Document Object Model)	traite un document XML ou HTML comme une arborescence dans laquelle chaque nœud représente une partie (balise) du document
Data Binding	processus qui permet aux utilisateurs de manipuler des éléments de page Web via un navigateur / utilise HTML dynamique / permet un meilleur affichage incrémentiel d'une page Web lorsque les pages contiennent une grande quantité de données
Jasmin	Framework de test
Karma	Gestionnaire de tâches pour les tests / test runner
Webpack	Bundler/convertisseur de modules gratuit / Permet de compiler TypeScript en JavaScript / Actualisation automatique à chaque modification.
TypeScript	langage de programmation développé et maintenu par Microsoft, qui introduit des fonctionnalités supplémentaires à JavaScript et peut également être compilé en JavaScript.
Tailwind CSS	framework CSS utilitaire-first qui permet de concevoir des interfaces rapidement en utilisant des classes CSS préconstruites directement dans le HTML ou les templates Angular.
Laravel	framework PHP open-source basé sur le paradigme MVC (Model-View-Controller)

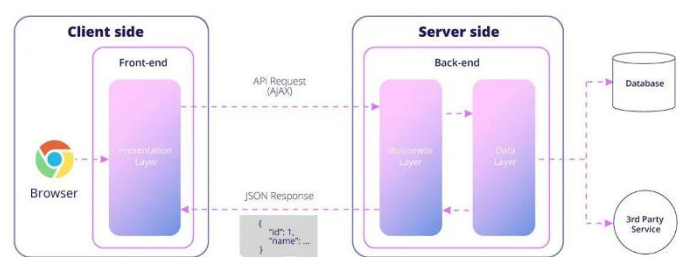
SSR / App classique



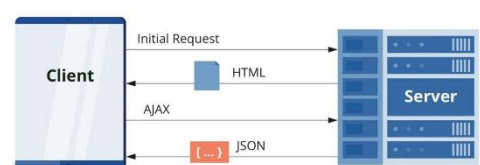
Traditional Page Lifecycle



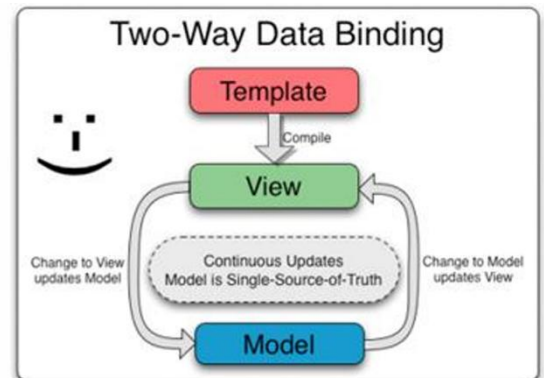
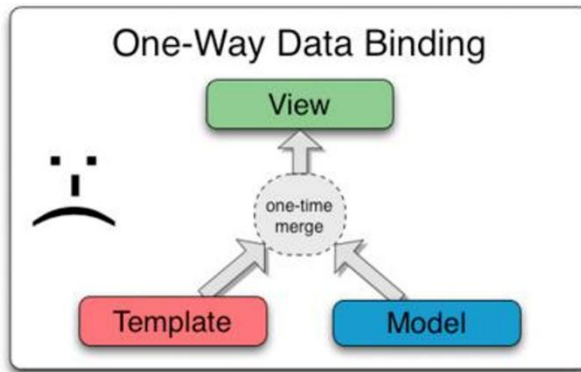
SPA / one page App



SPA Lifecycle



DATA Binding

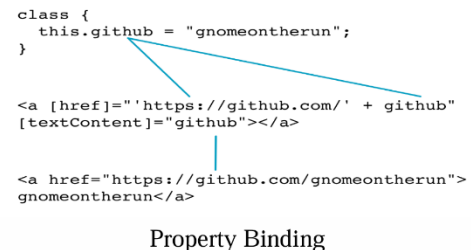
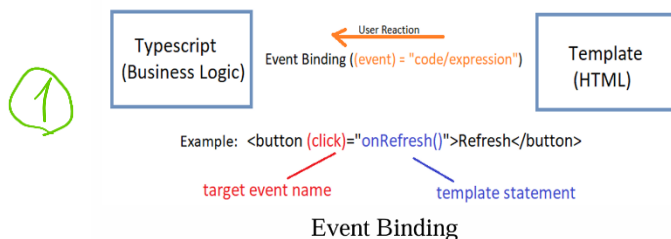


Angular utilise le 2 way data binding / liaison bidirectionnelle.

Mise à jour synchrone modèle ↔ interface

Le MVC avec Angular :

Modules	conteneur pour un groupe de composants, services, directives et canaux ⇔ package / AppModule est le module racine.
Component	représente une portion de l'interface / Contient fichier.ts (Classe avec des données, des méthodes = pivot entre le template et le service), template.html, style.css (⇔ controller + viewModel en MVC)
Template	Fichier.html enrichi avec des data- bindings (event-binding et property-binding) ¹ et directives.
Service	classe qui contient de la logique partagée entre plusieurs composants, utilisé pour accéder aux données (API), partager des fonctions communes, gérer l'état de l'app.



Installations : commandes importantes

<code>npm install -g typescript</code>	Installer typescript
<code>tsc -v</code>	Tester version typescript
<code>npm install -g @angular/cli@13.3.3</code>	Installer Angular CLI
<code>ng --version</code>	Tester version angular CLI
<code>ng new nom_projet</code>	Créer un projet WebApp
<code>ng serve</code>	Lancement projet
<code>tsc name_file.ts</code>	Compiler le ts en js
<code>node name_file.ts</code>	Lancer le fichier js compilé
<code>ng generate component comp_name</code>	créer un composant

ng generate pipe mypipe	Créer un pipe personnalisé
ng generate service serviceName	Création d'un service
ng generate guard auth	générer un Auth Guard
npm install -g json-server	installer Json Server

Structure d'un projet Angular

/e2e/	Test du site web simulant le comportement du user
/node_modules/	bibliothèques tierces installées ici à l'aide de npm install
/src/	Code source / tout les travaux
/app/	Modules et composants
/assets/	Éléments statiques : Images / icônes / styles
/environments/	fichiers de configuration spécifiques à l'environnement (prod/dev)
browserslist	Utilisé lors du build pour ajuster le css, js pour le navigateur
Favicon.ico	Icone angular
index.html	fichier HTML principal
karma.conf.js	fichier de configuration de Karma
main.ts	fichier de démarrage principal à partir duquel l' AppModule est amorcé
polyfills.ts	Permet à Angular de fonctionner sur des navigateurs non standard
styles.css	fichier de feuille de style globale pour le projet
tsconfig*.json	fichiers de configuration pour TypeScript
angular.json	configurations pour CLI
package.json	informations de base du projet (nom, description et dépendances)
tslint.json	fichier de configuration de TSLint (un outil d'analyse statique)

TypeScript

Fonctionnalités supplémentaires à JavaScript :

Typage strict / typage statique	<pre>let nom: string = "Alice"; let age: number = 25;</pre>
Interfaces	<pre>interface Personne { nom: string; age: number; } let user: Personne = { nom: "Bob", age: 30 };</pre>
Classes	<pre>class Animal { constructor(public nom: string) {} parler() { console.log("Je suis un animal"); } }</pre>

Classes abstraites	<pre>abstract class Forme { abstract aire(): number; }</pre>
Génériques ⇔ templates	<pre>function identite<T>(valeur: T): T { return valeur; } let result = identite<string>("Salut");</pre>
Décorateurs	<pre>@Component({ selector: 'app-test', template: '<h1>Hello</h1>' }) export class TestComponent {}</pre>
Enumérations	<pre>enum Direction { Haut, Bas, Gauche, Droite } let d: Direction = Direction.Haut;</pre>
Fonctionnalité orienté objects	Encapsulation / héritage / interfaces et abstractions / constructeurs ...
Inférence de type	<pre>let nom = "Alice"; // TypeScript comprend que c'est une string</pre>
Plateformes croisées	Le code TypeScript est transpilé en JavaScript, donc fonctionne partout (navigateurs, Node.js, etc.).
IntelliSense	aide intelligente dans l'éditeur (comme VS Code)
ES6 Features	une version majeure de JavaScript introduisant de nombreuses améliorations modernes. TypeScript en hérite complètement.

TypeScript Code (classes, interfaces, modules, types) -> compilation / transpiling
(typescript compiler, target : ES3, ES5, ES6) -> Vanilla JS Code (javascript file)

Déclaration d'une variable :

keyword **variable_Name** : **Type** = **Value**;

Keyword = var, const ou let

	Portée	Réaffectable	Redéclarable
var	fonction	oui	oui
let	bloc	oui	non
const	bloc	non	non

Portée fonction/ bloc : existe uniquement à l'intérieur de la fonction/bloc

Opérateurs

- ❑ Opérateurs arithmétiques: +, -, *, /, %, ++, --
- ❑ Opérateurs logiques: <, >, <=, >=, ==, !=
- ❑ Opérateurs relationnels: &&(and), || (or), ! (not)
- ❑ Opérateurs d'affectation: =, +=, -=, *=, /=
- ❑ Opérateur conditionnel: Test ? expr1 : expr2
- ❑ Opérateur de chaîne de caractères: Concatenation +
- ❑ Opérateur de type: typeof var

DataType en TypeScript

number

- Décimal,
- Octal,
- Binaire,
- Hexadécimal,

```
let d : number = 10;
let o : number = 0o7;
let b : number = 0b10;
let h : number = 0xFF;
```

.toExponential()	Retourne une chaîne représentant le nombre en notation scientifique
.toFixed(n)	Formate le nombre avec n chiffres après la virgule
.toLocaleString()	Retourne une chaîne localisée (avec des séparateurs adaptés à la langue/culture).
.toPrecision(n)	Retourne une chaîne avec un nombre total de chiffres significatifs égal à n
.toString()	Convertit le nombre en chaîne de caractères .
.valueOf()	Retourne la valeur primitive du nombre (utile si le nombre est un objet Number).
.typeof	retourne le type de la variable ou de la valeur.

String

On peut utiliser ', ", ou `

.charCodeAt(index: number)	Renvoie le code Unicode du caractère à l'index donné.
.String.fromCharCode(...codes: number[])	Convertit des codes Unicode en chaîne de caractères .
.includes(texte: string)	Renvoie true si le texte est présent dans la chaîne.
.indexOf(texte: string)	Renvoie la position de la première occurrence du texte.
.lastIndexOf(texte: string)	Renvoie la dernière position d'une occurrence.
.toLowerCase()	Convertit la chaîne en minuscules .

. toUpperCase()	Convertit la chaîne en majuscules .
. localeCompare(autre: string)	Renvoie 0 si égales, 1 si la première chaîne > la 2ème, -1 si inférieur
.substring(start, end?)	Extrait une portion de la chaîne (du caractère start à end-1).
. trim()	Supprime les espaces en début et fin de chaîne.
. split(séparateur)	Découpe la chaîne en tableau de sous-chaînes
. replace(ancienne, nouvelle)	Remplace une sous-chaîne par une autre.

Boolean

Boolean est un type mais aussi une fonction (Boolean()) :

```
console.log(Boolean(false)) // returns false
console.log(Boolean(true)) // returns true
console.log(Boolean("AAAbbb")) // returns true
console.log(Boolean(100.50)) // returns true
console.log(Boolean(NaN)) // returns false
console.log(Boolean(undefined)) // returns false
```

Any

Permet d'accepter n'importe quel type/À utiliser quand le type est inconnu ou variable

Unknown

Comme any mais force une vérification de type avant utilisation

Array

tableau de valeur de type nombre

```
let myVariable : number [] = [1,2];
myVariable.push(3);
console.log(myVariable);
```

→ permet d'ajouter des éléments

Ou aussi :

```
let arr: Array<string> = [];
```

Tuple

tableau de valeurs **de types différents** à des positions précises

```
let laptopDetails: [number, string, string][];
laptopDetails = [[5000, "Dell", "Intel CORE i3"],
                  [2000, "Lenovo", "Intel CORE i5"],
                  [1000, "HP", "Intel CORE i5 with Graphics"]];
```

Enum

Type de données avec des valeurs nommées constantes (pas modifiable)

```
enum Color {Red = 0, Green = 1, Blue = 2};
```

Fonction et classes en TypeScript

```
function disp_details(id:number,name:string,mail_id?:string) : string{
    console.log("ID:", id);
    console.log("Name",name);

    if(mail_id!==undefined)
        console.log("Email Id",mail_id);
    return 'Done'
}
disp_details(123,"John");
disp_details(111,"mary","mary@xyz.com");
```

→ paramètre facultatif

→ type de retour

```
let addition = (a: number, b: number): number =>
    return a + b;
};
```

fonction anonyme/lambda/
arrow function

une fonction typée (): never ? retourne une erreur ou boucle infiniment

interfaces :

```
interface IPerson {
    firstName:string,
    lastName:string,
    sayHi: ()=>string
}

let person_1:IPerson = {
    firstName:"Tom",
    lastName:"Hanks",
    sayHi: ():string =>{return "Hi, I'm Tom"}
}

console.log("Person 1 Details ");
console.log(person_1.firstName);
console.log(person_1.lastName);
console.log(person_1.sayHi());
```

définit la syntaxe à laquelle toute entité doit adhérer. Les interfaces définissent des propriétés, des méthodes et des événements, qui sont les membres de l'interface.

Différences avec type :

interface peut être étendue, type non
type est plus puissant pour composer les types

On peut combiner plusieurs

interface avec **extends**.

Bonus

appliquer une interface à une classe :

class MyClass implements MyInterface

Partial<T> :

rendre tous les champs de T optionnels

keyof :

Renvoie le type des clés d'un objet

Record<string, number>

Crée un objet avec des clés string et des valeurs number

Type

Mot clé pour créer un alias de type

Exclude<T, U>

Exclut de T les types assignables à U

ReturnType<typeof maFonction>

Récupère le type de retour de la fonction

Infer

Sert à Dédire un type automatiquement

Pick<T, K>

Extraire certains champs du type T

NonNullable<T>

Interdire null et undefined dans un type

classes

```
class Person {  
}
```

TypeScript

```
var Person = /** @class */ (function () {  
    function Person() {  
    }  
    return Person;  
})();
```

JavaScript

```

class Car {
  //field
  engine:string;

  //constructor
  constructor(engine:string) {
    this.engine = engine
  }

  //function
  disp():void {
    console.log("Engine is : "+this.engine)
  }
}

let Zoe = new Car("ZE.Elec");
console.log(typeof Zoe);
Zoe.disp();

```

modificateur d'accès : **public**, **private**, **protected**. **Extends** pour héritage, **W** pour classe abstraites, **static** devant une propriété fait qu'elle appartient à la classe elle-même, pas aux instances.

Composants Angular

- Modèle HTML
- Classe typescript
- Sélecteur qui définit comment le composant est utilisé dans un modèle
- Style CSS

Décorateur

modèles de conception utilisés pour isoler la modification ou la décoration d'une classe sans modifier le code source / fonctions qui permettent de modifier un service, une directive ou un filtre avant son utilisation

@Component()	Définit une classe comme composant Angular
@Directive()	Crée une directive personnalisée
@Injectable()	Rend une classe injectable (service, etc.)
@NgModule()	Définit un module Angular
@Input()	Rend une propriété accessible depuis le parent
@Output()	Permet d'émettre des événements au parent
@ViewChild()	Récupère un élément du template enfant
@ContentChild()	Récupère un contenu transmis via ng-content
@HostListener()	Réagit à un événement DOM (click, scroll...)

@HostBinding()	Lie une propriété à un attribut HTML
@Inject()	Injecte un service manuellement (rare)
@Optional()	Rend une dépendance facultative
@Self(), @SkipSelf(), @Host()	Contrôle le niveau d'injection

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-message',
  template: `<p>{{texte}}</p><button (click)="envoyer()">Envoyer</button>`
})
export class MessageComponent {
  @Input() texte: string = '';
  @Output() envoi = new EventEmitter<string>();

  envoyer() {
    this.envoi.emit(this.texte);
  }
}
```

Vert : décorateur de classe

Rouge : Décorateur de propriété

noir : décorateur de méthode ou paramètre

@Input

permet à un composant enfant de recevoir des données depuis son composant parent.

1. Composant fils (enfant.component.ts)

```
ts

import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-enfant',
  template: `<p>Bonjour {{ prenom }} !</p>`
})
export class EnfantComponent {
  @Input() prenom!: string;
}
```

 Copier  Modifier

◆ Grâce à @Input(), la variable prenom devient accessible depuis le parent.

2. Composant parent (parent.component.ts)

ts

 Copier  Modifier

```
@Component({
  selector: 'app-parent',
  template: `<app-enfant [prenom]="nomEnfant"></app-enfant>`
})
export class ParentComponent {
  nomEnfant = 'Alice';
}
```

◆ Le parent passe la valeur 'Alice' à l'enfant via la **liaison de propriété** [prenom] .

Lorsque tu veux passer un objet entier comme @Input() dans un composant enfant (et pas juste une simple string ou un nombre), Angular le gère très bien :



ParentComponent.ts

ts

 Copier  Modifier

```
@Component({
  selector: 'app-parent',
  template: `<app-user-card [user]="utilisateur"></app-user-card>`
})
export class ParentComponent {
  utilisateur = {
    nom: 'Alice',
    age: 25,
    email: 'alice@email.com'
  };
}
```



UserCardComponent.ts (Enfant)

ts

 Copier  Modifier

```
@Component({
  selector: 'app-user-card',
  template: `<p>{{ user.nom }} - {{ user.age }} ans</p>`
})
export class UserCardComponent {
  @Input() user!: { nom: string; age: number; email: string };
}
```

Sélecteur

propriété du décorateur @Component qui détermine comment et où utiliser un composant dans un template HTML / Indique la balise HTML personnalisée

```
@Component({  
  selector: 'app-mon-composant',  
  templateUrl: './mon-composant.component.html'  
})  
export class MonComposant { }
```

Type de sélecteur	Syntaxe	Utilisation dans le HTML
Élément	'app-truc'	<app-truc></app-truc>
Classe	'ma-classe'	<div class="ma-classe"></div>
Attribut	'[ma-directive]'	<div ma-directive></div>
Combiné	'div[ma-directive]'	<div ma-directive> (seulement sur)

Bootstrap

Intégration de Bootstrap dans un projet Angular

- Méthode 1 : via CDN (dans index.html)

```
<link rel="stylesheet"  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
```

- Méthode 2 : via npm

```
npm install bootstrap
```

Puis dans angular.json :

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css"  
,  
  "scripts": [  
    "node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"  
  ]
```

Structure principale de Bootstrap

1. Système de grille (Grid System)

Permet de créer des mises en page en colonnes responsives :

```
<div class="container">

  <div class="row">

    <div class="col-6">Colonne 1</div>

    <div class="col-6">Colonne 2</div>

  </div>

</div>
```

On peut utiliser :

- .container ou .container-fluid
- .row pour les lignes
- .col, .col-6, .col-md-4 pour les colonnes

2. Classes utilitaires (spacing, couleurs, etc.)

Classe	Effet
mt-3, mb-2	margin top/bottom
p-2, px-4	padding
bg-primary	fond bleu Bootstrap
text-white	texte blanc
d-flex, justify-content-center	mise en page avec flexbox

3. Composants visuels prêts à l'emploi

- **Bouton :**

```
<button class="btn btn-primary">Valider</button>
```

- **Carte (card) :**

```
<div class="card">
```

```
<div class="card-body">

  <h5 class="card-title">Titre</h5>

  <p class="card-text">Contenu...</p>

</div>

</div>
```

- **Navbar (barre de navigation) :**

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">

  <a class="navbar-brand" href="#">MonApp</a>

</nav>
```

- **Modale :**

```
<button data-bs-toggle="modal" data-bs-target="#exempleModal">Ouvrir</button>

<div class="modal fade" id="exempleModal">

  <div class="modal-dialog">

    <div class="modal-content">

      ...

    </div>

  </div>

</div>
```

Directives

aident à manipuler le DOM

- Component Directive -> C'est juste les composants
- Attribute Directive
- Structural Directive

Structural Directive

ajoute, modifie ou supprime dynamiquement des éléments HTML du DOM.

*ngIf : Affiche ou cache un élément en fonction d'une condition

```
<p *ngIf="estConnecte">Bienvenue, utilisateur !</p>
```

// x! sert à dire au compilateur que x n'est pas null ou undefined

Avec else :

```
<p *ngIf="estConnecte; else pasConnecte">Bienvenue !</p>
```

```
<ng-template #pasConnecte>
```

```
  <p>Veuillez vous connecter</p>
```

```
</ng-template>
```

***ngFor : Répète un élément pour chaque item d'une liste**

```
<ul>
```

```
  <li *ngFor="let fruit of fruits">{{ fruit }}</li>
```

```
</ul>
```

Tu peux aussi récupérer l'index :

```
<li *ngFor="let fruit of fruits; let i = index">{{ i + 1 }} - {{ fruit }}</li>
```

***ngSwitch : Affiche un seul élément parmi plusieurs selon une valeur donnée**

```
<div [ngSwitch]="role">
```

```
  <p *ngSwitchCase="'admin'">Bienvenue Admin</p>
```

```
  <p *ngSwitchCase="'user'">Bienvenue Utilisateur</p>
```

```
  <p *ngSwitchDefault>Rôle inconnu</p>
```

```
</div>
```

Attribut Directives

manipulent le DOM en modifiant son comportement et son apparence / ne créent pas ni ne suppriment d'éléments dans le DOM.

NgClass

Permet d'**ajouter dynamiquement des classes CSS** à un élément.

```
<div [ngClass]="{ 'rouge': estErreur, 'vert': estValide }"></div>
```

Ou


```
<li class="list-group-item" [ngClass]='{"fontColor size20 backgroundColor" :  
product.price > 10}'>Price :
```

ngStyle

Permet d'**appliquer dynamiquement des styles CSS** inline.

```
<div [ngStyle]="{ 'color': 'red', 'font-size': taille + 'px' }"></div>
```

Ou

```
<li class="list-group-item" [ngStyle]="{'background': product.price>10 ? 'red' :  
'green'}">Price : {{product.price}} $ </li>
```

ngModel

Lien bidirectionnel entre une **propriété du composant** et un **champ de formulaire**.

```
<input [(ngModel)]="nom">
```

```
<p>Bonjour {{ nom }}</p>
```

ngModelChange

Détecte un **changement de valeur** d'un champ avec ngModel.

```
<input [(ngModel)]="nom" (ngModelChange)="onChange($event)">
```

ngClass + ngStyle ensemble

Ils peuvent être combinés pour gérer les **styles complexes conditionnels**.

```
<div [ngClass]="{'important': estImportant}"  
      [ngStyle]="{ 'font-weight': estImportant ? 'bold' : 'normal' }">  
</div>
```

Directives supplémentaires

Directive	Description courte
[disabled]	Active/désactive un bouton ou champ ([disabled]="true")
[required]	Rend un champ obligatoire
[checked]	Coche/décoche un champ
[readonly]	Rend un champ non modifiable
[hidden]	Masque un élément sans le retirer du DOM ([hidden]="true")

Directive	Description courte
[attr.*]	Ajoute un attribut personnalisé (ex : [attr.aria-label])
Événement	Description courte
(click)	Gère un clic de souris
(input)	Déclenché lors de la saisie
(change)	Lorsqu'une valeur change (souvent pour des <select>)
(keyup) / (keydown)	Lorsqu'une touche est pressée

Tubes / Pipes

permet de **transformer des données dans le template HTML**, sans changer leur valeur dans le TypeScript / fonctions simples à utiliser dans les expressions de modèle pour accepter une valeur d'entrée et renvoyer une valeur transformée

{{ valeur | nomDuPipe }}

uppercase	Met le texte en MAJUSCULES	` {{ nom
lowercase	Met le texte en minuscules	` {{ nom
titlecase	Met en majuscule la première lettre de chaque mot	` {{ titre
date	Formate une date	` {{ aujourdHui
currency	Formate une valeur en monnaie locale	` {{ prix
percent	Affiche un pourcentage	` {{ 0.25
decimal	Formate un nombre décimal	` {{ 3.14159
json	Affiche un objet JSON formaté	` {{ objet
slice	Coupe un tableau ou une chaîne	` {{ phrase
async	Affiche la valeur d'un Observable ou Promise	` {{ monObservable\$

Créer des pipes personnalisés

1. Générer le pipe :

ng generate pipe nomDuPipe

2. Exemple : créer un pipe inverse (inverse une chaîne)

```
import { Pipe, PipeTransform } from '@angular/core';
```

```
@Pipe({ name: 'inverse' })
```

```
export class InversePipe implements PipeTransform {
```

```
  transform(value: string): string {
```

```
    return value.split('').reverse().join('');
```

```
  }
```

```
}
```

3. Utilisation :

```
<p>{{ 'Angular' | inverse }}</p> <!-- Affiche: ralugnA -->
```

Service

classe TypeScript qui contient de la **logique métier** ou des **fonctions réutilisables** (comme appels API, stockage de données, gestion utilisateur, etc.).

Pourquoi utiliser des services ?

Sans service

Code dupliqué dans chaque composant

Maintenance difficile

Aucune séparation de logique

Avec service

Code centralisé et réutilisable

Maintenance simplifiée

Respect de l'architecture MVC/Angular

ng generate service nom-du-service -> création d'un service

exemple: un service qui fournit un message

```
import { Injectable } from '@angular/core';
```

```
@Injectable({
```

```
  providedIn: 'root' // rend le service disponible partout (singleton)
```

```
})
```

```
export class MessageService {  
  getMessage(): string {  
    return 'Bonjour depuis le service !';  
  }  
}
```

Utilisation dans un composant

Dans mon-composant.component.ts :

```
import { Component } from '@angular/core';  
import { MessageService } from './message.service';
```

```
@Component({  
  selector: 'app-mon-composant',  
  template: `<p>{{ message }}</p>`  
})  
export class MonComposant {  
  message: string = "";  
  
  constructor(private msgService: MessageService) {}  
  
  ngOnInit() {  
    this.message = this.msgService.getMessage();  
  }  
}
```

Grâce à l'**injection de dépendance**, Angular fournit automatiquement le service via le constructeur.

Que peut faire un service ?

Fonctionnalité	Exemples concrets
Logique métier	Calculs, validations, règles d'entreprise
Communication HTTP	Appels API avec HttpClient
Stockage partagé	Mémoriser des données entre composants
Accès local/sessionStorage	Gérer des tokens, préférences
Gestion d'état (mini Redux)	Conserver l'état d'un panier, utilisateur
Observables et RxJS	Événements asynchrones, temps réel

Event Binding

Nous l'utilisons pour effectuer une action dans le composant lorsque l'utilisateur effectue une action comme cliquer sur un bouton, modifier l'entrée, etc. dans la vue / **lier un événement du DOM** à une **méthode** dans le **composant TypeScript**.

Syntaxe :

```
<balise (evenement)="methodeDuComposant()"></balise>
```

```
<button (click)="update()">Submit</button>
```

nom d'événement cible entre parenthèses à gauche et une déclaration de modèle entre guillemets à droite.

Événements DOM les plus utilisés

Événement Description

(click)	Clic sur un bouton ou élément
(dblclick)	Double-clic
(input)	Changement de texte dans un champ
(change)	Changement dans un <select>
(keyup)	Une touche est relâchée
(keydown)	Une touche est pressée
(mouseover)	Souris passe sur l'élément

Événement Description

(submit) Soumission d'un formulaire

Avantages

- Réactif et simple
- Permet de **lier l'interface utilisateur à la logique** facilement
- Compatible avec tous les événements HTML natifs

\$event : contient les informations détaillées sur un événement

Two Way Binding / liaison bidirectionnelle

peut être obtenue à l'aide d'une directive ngModel

permet de **lier une propriété du composant à un champ HTML**, de façon à ce que :

- Quand l'utilisateur modifie l'input, la variable TypeScript est mise à jour automatiquement
- Quand la variable change dans le code, le champ HTML se met à jour aussi

Syntaxe :

```
<input [(ngModel)]="nom">
```

Exemple complet

Composant (app.component.ts)

```
export class AppComponent {  
  nom: string = 'Alice';  
}
```

Template HTML (app.component.html)

```
<input [(ngModel)]="nom">  
<p>Bonjour {{ nom }} !</p>
```

Avantages du Two-Way Binding

Avantage	Détail
Synchronisation automatique	Pas besoin de gérer manuellement les événements
Moins de code	Pas besoin de @ViewChild ni de event.target.value
Idéal pour les formulaires	Saisie, modification, validation dynamique

Routage

mécanisme permettant de naviguer entre différentes vues de composants.

bibliothèque @angular/router

Étapes pour mettre en place le routage

1. Créer les composants

2. Définir les routes dans app-routing.module.ts

```
const routes: Routes = [
  { path: '', component: AccueilComponent },
  { path: 'contact', component: ContactComponent },
  { path: '**', component: Erreur404Component } // route de secours
];
```

```
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
```

3. Ajouter <router-outlet> dans app.component.html

```
<nav>
  <a routerLink="/">Accueil</a>
  <a routerLink="/contact">Contact</a>
</nav>
```

<router-outlet></router-outlet>

Les balises Angular du routage

lise/directive	Rôle
<router-outlet>	Point où s'affichent les composants liés à l'URL
[routerLink]="['/chemin']"	Crée un lien de navigation interne
routerLinkActive="..."	Ajoute une classe CSS si le lien est actif
pathMatch: 'full'	Oblige à matcher l'URL exactement (utile pour la racine)

Router-outlet : espace réservé qui est utilisé pour charger dynamiquement les différents composants en fonction du composant activé ou de l'état actuel de la route.

RouterLink : directive d'Angular intégrée qui permet à l'application de créer un lien vers des routes spécifiques dans l'application.

Route Parameters : permettent de transmettre des données dynamiques dans l'URL (par exemple un id, un nom, etc.) à un composant.

Définir une route paramétrée : { path: 'produit/:id', component: X }

Créer un lien : <a [routerLink]="['/produit', id]">

Récupérer le paramètre : this.route.snapshot.paramMap.get('id')

Child Routes/ routes enfants :

route définie dans une autre route, appelée route parente.

permet d'afficher plusieurs composants dans un même layout, par exemple :

- /admin/utilisateurs
- /admin/statistiques
- /profil/infos
- /profil/modifier

URL	Composant affiché
/admin	AdminComponent avec redirection
/admin/utilisateurs	AdminComponent + UtilisateursComponent
/admin/statistiques	AdminComponent + StatistiquesComponent

Authgard

service spécial qui décide si une route peut être activée ou non, en fonction de règles comme :

- L'utilisateur est-il connecté ?
- A-t-il un rôle précis (admin, prof, etc.) ?

Interface Angular Utilité spécifique

CanActivate	Autorise ou non l'accès avant d'entrer dans une route
CanActivateChild	Comme CanActivate, mais pour les routes enfants
CanLoad	Empêche le chargement d'un module entier (lazy loading)
CanDeactivate	Vérifie avant de quitter une page (ex : formulaire non sauvegardé)

1. **générer un Guard** : ng generate guard auth
2. **Implémenter la logique d'autorisation**

```
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}

  canActivate(): boolean {
```

```

const estConnecte = !!localStorage.getItem('token');

if (estConnecte) {
  return true;
} else {
  this.router.navigate(['/login']);
  return false;
}
}
}

```

3. Protéger une route dans app-routing.module.ts

```

import { AuthGuard } from './auth.guard';

const routes: Routes = [
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [AuthGuard]
  }
];

```

Exemple pour vérifier le role :

```

canActivate(): boolean {
  const role = localStorage.getItem('role');
  if (role === 'admin') {
    return true;
  } else {
    this.router.navigate(['/forbidden']);
    return false;
  }
}

```

```
}  
  
}
```

Formulaire / form

l'interface pour interagir avec le modèle pour ajouter ou modifier les attribut

ChangeDetectorRef : détecte les changements dans un formulaire

Critère	Template Driven Form	Reactive Form / Model Driven Form
Gestion du formulaire	Dans le template HTML	Dans le fichier TypeScript
Module requis	FormsModule	ReactiveFormsModule
Complexité	Simple, déclaratif	Plus complexe, mais plus puissant
Validation	Via les attributs HTML (required, etc.)	Définie en code avec des Validators
Réactivité	Faible	Très bonne (via observables valueChanges)
Tests unitaires	Difficiles	Faciles
Structure du formulaire	Automatique avec [(ngModel)]	Déclarée explicitement en TypeScript

Template-Driven Forms

Avantages :

- Simple et rapide à mettre en place
- Idéal pour les **petits formulaires**
- Syntaxe naturelle et proche du HTML natif
- Bon pour des débutants

Inconvénients :

- **Difficile à tester**

- Moins de contrôle sur la logique métier
- Gestion des erreurs plus compliquée
- Pas adapté aux formulaires dynamiques
- Moins réactif

Reactive / Model-Driven Forms

Avantages :

- **Haute testabilité**
- Plus de **contrôle programmatique**
- Parfait pour les **formulaires complexes ou dynamiques**
- Suivi en temps réel (observable, valueChanges)
- Validation très personnalisable
- Meilleure séparation logique (HTML / logique métier)

Inconvénients :

- Plus **verbeux**
- Courbe d'apprentissage un peu plus élevée
- Nécessite plus de code pour les petits cas

FormControl

classe Angular utilisée pour gérer un champ unique de formulaire en mode Reactive Form.

1. Importer FormControl

Avant de l'utiliser, il faut importer :

```
import { FormControl } from '@angular/forms';
```

2. Créer un FormControl simple

```
nom = new FormControl('Julie');
```

Ce champ contient la valeur "Julie" par défaut.

3. Lier à l'HTML

```
<input [formControl]="nom">
```

```
<p>Bonjour {{ nom.value }}!</p>
```

La valeur se met à jour **automatiquement** en fonction de ce que tape l'utilisateur.

4. Ajouter une validation

```
import { Validators } from '@angular/forms';
```

```
nom = new FormControl('', [Validators.required, Validators.minLength(3)]);
```

```
<input [formControl]="nom">
```

```
<div *ngIf="nom.invalid && nom.touched">
```

Le nom est requis (au moins 3 caractères)

```
</div>
```

5. Vérifier l'état du champ

Méthode / propriété Rôle

nom.value	Donne la valeur actuelle
-----------	--------------------------

nom.valid	True si valide
-----------	----------------

nom.invalid	True si invalide
-------------	------------------

nom.touched	True si l'utilisateur a touché le champ
-------------	---

nom.dirty	True si l'utilisateur a modifié la valeur
-----------	---

nom.errors	Liste des erreurs de validation (ex: required)
------------	--

6. Mettre à jour ou réinitialiser

```
this.nom.setValue('Lucas');
```

```
this.nom.reset(); // vide le champ
```

7. S'abonner aux changements

```
this.nom.valueChanges.subscribe(val => {  
  console.log('Nouvelle valeur :', val);  
});
```

Résumé des méthodes utiles

Méthode	Description
setValue(val)	Met à jour complètement la valeur
patchValue(val)	Met à jour partiellement (surtout pour les groupes)
reset()	Réinitialise le champ (valeur vide + état vierge)
disable() / enable()	Active ou désactive le champ
setValidators([...])	Modifie les validateurs
updateValueAndValidity()	Recalcule l'état du champ (à appeler si modifié)

FormGroup

structure qui regroupe plusieurs FormControl ensemble pour :

- gérer des formulaires complets (avec plusieurs champs),
- valider l'ensemble du formulaire,
- accéder aux valeurs champ par champ.

1. Importer FormGroup et FormControl

```
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

2. Créer un FormGroup

Exemple simple :

```
formulaire = new FormGroup({  
  nom: new FormControl("", Validators.required),  
  email: new FormControl("", [Validators.required, Validators.email])  
});
```

➡ Ce formulaire contient 2 champs : nom et email.

3. Lier le FormGroup au HTML

```
<form [formGroup]="formulaire" (ngSubmit)="onSubmit()">
  <input formControlName="nom" placeholder="Nom">
  <input formControlName="email" placeholder="Email">
  <button type="submit">Envoyer</button>
</form>
```

4. Accéder aux données dans le composant

```
onSubmit() {
  if (this.formulaire.valid) {
    console.log(this.formulaire.value); // { nom: '...', email: '...' }
  }
}
```

Accès à un champ spécifique

```
this.formulaire.get('nom')?.value;
this.formulaire.get('nom')?.valid;
this.formulaire.get('nom')?.errors;
```

5. Méthodes utiles de FormGroup

Méthode / propriété	Description
form.value	Renvoie les valeurs des champs (objet)
form.get('nom')	Accède à un champ
form.valid / form.invalid	True / false selon la validité globale
form.reset()	Réinitialise tous les champs
form.setValue({ ... })	Modifie tous les champs
form.patchValue({ ... })	Modifie une partie des champs
form.disable() / form.enable()	Active / désactive tout le formulaire
form.valueChanges.subscribe()	Réagit aux changements de tout le groupe

Validator

fonction qui prend un champ (FormControl) et renvoie :

- null si le champ est valide
- un objet erreur si le champ est invalide

Validator	Rôle
Validators.required	Le champ est obligatoire
Validators.minLength(n)	Doit avoir au moins n caractères
Validators.maxLength(n)	Ne doit pas dépasser n caractères
Validators.email	Doit respecter le format email (truc@domaine.com)
Validators.pattern(regex)	Doit respecter une expression régulière donnée
Validators.nullValidator	Toujours invalide (utile pour forcer une erreur)

Résumé des états utiles

Propriété	Signification
control.valid	Le champ est valide
control.invalid	Le champ est invalide
control.errors	Contient les erreurs de validation
control.hasError('required')	True si le champ est vide
control.hasError('minlength')	True si la longueur est trop courte

Nested FormGroup / FormGroup imbriqué

groupe de champs à l'intérieur d'un autre groupe.

Cela permet :

- une meilleure organisation
- des validations ciblées
- un accès plus structuré aux données

FormArray

tableau de FormControl, FormGroup ou même de FormArray.

Il est utilisé pour ajouter, supprimer et manipuler dynamiquement des champs répétés dans un formulaire.

1.Import nécessaire

```
import { FormArray, FormControl, FormGroup, FormBuilder, Validators } from '@angular/forms';
```

2. Exemple simple : une liste de compétences

Objectif :

```
{  
  "competences": ["Angular", "TypeScript", "Node.js"]  
}
```

Déclaration dans component.ts

```
form = new FormGroup({  
  competences: new FormArray([  
    new FormControl('Angular', Validators.required)  
  ])  
});  
  
get competences() {  
  return this.form.get('competences') as FormArray;  
}  
  
ajouterCompetence() {  
  this.competences.push(new FormControl("", Validators.required));  
}  
  
supprimerCompetence(index: number) {  
  this.competences.removeAt(index);  
}
```

// as sert à faire un cast de type

HTML

```

<form [formGroup]="form" (ngSubmit)="onSubmit()">
  <div formArrayName="competences">
    <div *ngFor="let c of competences.controls; let i = index">
      <input [formControlName]="i" placeholder="Compétence {{ i + 1 }}">
      <button type="button" (click)="supprimerCompetence(i)">Supprimer</button>
    </div>
  </div>
  <button type="button" (click)="ajouterCompetence()">Ajouter compétence</button>
  <button type="submit">Envoyer</button>
</form>

```

Affichage dans onSubmit()

```

onSubmit() {
  console.log(this.form.value);

  // => { competences: ["Angular", "TypeScript", "Node.js"] }
}

```

Design Pattern

solutions génériques aux défis courants de la conception logicielle, indépendantes des langages de programmation.

Creational Design Pattern / patrons de conception de création

solutions standards à des problèmes liés à la création d'objets dans un programme. Ils permettent de :

- masquer la logique d'instanciation,
- centraliser la création d'objets complexes,
- garantir des contraintes (comme l'unicité),
- et favoriser l'extensibilité.

Pattern	Description courte
Singleton	Garantit qu'il n'existe qu'une seule instance d'une classe

Pattern	Description courte
---------	--------------------

Factory Method	Délègue la création à une méthode d'une sous-classe
----------------	---

Abstract Factory	Fournit une famille d'objets liés sans spécifier leurs classes
------------------	--

Builder	Crée un objet complexe étape par étape
---------	--

Prototype	Crée des objets par copie d'un prototype existant
-----------	---

Structural Design Patterns

répondent à cette question :

"Comment structurer les classes et objets pour qu'ils travaillent bien ensemble ?"

Ils se concentrent sur :

- la composition d'objets et de classes,
- la réutilisabilité et la modularité,
- le fait de cacher la complexité.

Pattern	Description courte
---------	--------------------

Adapter	Rend compatibles des interfaces incompatibles
---------	---

Bridge	Sépare l'abstraction de l'implémentation pour les faire évoluer indépendamment
--------	--

Composite	Gère une hiérarchie d'objets (arborescence) de manière uniforme
-----------	---

Decorator	Ajoute dynamiquement des fonctionnalités à un objet
-----------	---

Facade	Fournit une interface simplifiée à un sous-système complexe
--------	---

Flyweight	Réutilise des objets partagés pour économiser de la mémoire
-----------	---

Proxy	Crée un objet représentant un autre objet pour contrôler son accès
-------	--

Behavioral Design Patterns

défini comment les objets interagissent entre eux, comment ils :

- échangent des informations,
- répartissent les responsabilités,

- réagissent à des événements ou des commandes,
- tout en restant découplés pour faciliter l'évolution du code.

Pattern Rôle principal

Observer	Réagit aux changements d'un autre objet
Strategy	Permet de changer dynamiquement d'algorithme
Command	Encapsule une action sous forme d'objet
Mediator	Centralise la communication entre plusieurs objets
State	Change le comportement d'un objet selon son état
Chain of Responsibility	Passe une requête à une chaîne d'objets jusqu'à traitement
Iterator	Parcourt une collection sans exposer sa structure interne
Template Method	Définit une structure d'algorithme tout en laissant des étapes personnalisables
Visitor	Applique des opérations sur des objets sans les modifier
Interpreter	Évalue des expressions dans un langage personnalisé
Memento	Sauvegarde et restaure un état interne d'un objet

Backend : Json Server

outil Node.js qui permet de :

- créer une API REST complète et fonctionnelle en quelques secondes,
- simuler un backend pour le développement front-end,
- utiliser un simple fichier .json comme base de données.

Installation :

`npm install -g json-server`

Exemple de fichier db.json

```
{
  "users": [
    { "id": 1, "nom": "Alice" },
    { "id": 2, "nom": "Bob" }
  ],
}
```

```
"posts": [  
  { "id": 1, "titre": "Hello", "contenu": "Premier post" }  
]  
}
```

Lancer le serveur

```
json-server --watch db.json
```

Endpoints REST générés automatiquement

Si tu as users et posts dans ton db.json, tu auras :

Méthode HTTP	URL	Action
GET	/users	Obtenir tous les users
GET	/users/1	Obtenir un user par ID
POST	/users	Créer un user
PUT	/users/1	Modifier complètement
PATCH	/users/1	Modifier partiellement
DELETE	/users/1	Supprimer un user

Avantages

- Facile à installer et à utiliser |
- Très utile pour le développement Angular |
- API REST standard générée automatiquement |
- Permet de tester sans serveur réel |
- Données persistées dans un fichier .json |

Limitations

- Pas de **logique métier complexe** (c'est juste un simulateur).
- Pas de vraie **authentification sécurisée** (mais possibilité de la simuler).
- Pas prévu pour la **production** (uniquement pour tests/devs).

http service

Data Streams

Lorsque tu utilises le HttpClient d'Angular pour faire des requêtes vers un backend (comme JSON Server), tu obtiens un flux de données (Observable) et non une valeur immédiate. Cela te permet de réagir aux données asynchrones, de chaîner des traitements, et de gérer les erreurs ou les chargements proprement.

Résumé des opérateurs utiles pour les data streams

Opérateur RxJS Rôle

map()	transforme chaque valeur du flux
filter()	ne garde que certaines valeurs
tap()	exécute une action sans modifier le flux (debug)
catchError()	intercepte les erreurs et retourne un flux alternatif
switchMap()	annule les requêtes précédentes quand une nouvelle arrive
mergeMap()	exécute toutes les requêtes en parallèle
take(n)	prend les n premières valeurs

- HttpClient = API Angular pour faire des appels http
- Chaque appel renvoie un Observable, donc un **data stream**
- Tu dois t'y **abonner** pour recevoir la réponse
- Tu peux **transformer** le flux avec pipe() et les opérateurs RxJS
- Très puissant pour les interfaces réactives (chargement, filtrage, erreurs...)

Observable

Flux de données asynchrone fourni par la bibliothèque RxJS, utilisée par Angular. Quand tu fais une requête HTTP avec HttpClient, Angular ne te renvoie pas directement les données, mais un Observable que tu peux observer (= t'abonner).

Pourquoi Angular utilise-t-il des Observable avec HttpClient ?

Avantage Explication

Asynchrone	Permet de ne pas bloquer l'interface pendant le chargement
Réactif	Peut réagir automatiquement à l'arrivée de nouvelles données
Flexible	Tu peux transformer, combiner ou filtrer les données
Annulable	Tu peux facilement annuler une requête (avec takeUntil par ex.)
Chaînable	Tu peux appliquer des pipe() et map(), filter(), catchError() etc.

Différence Observable vs Promise

Aspect	Observable	Promise
Valeurs multiples	oui	une seule
Annulation possible	oui (unsubscribe())	non
Lazy (à la demande)	oui	oui
Opérateurs RxJS	oui (map, filter, mergeMap)	non

En résumé

- HttpClient.get() retourne un Observable<T>
- Tu dois **t'abonner** pour que la requête parte (.subscribe())
- Tu peux **chaîner des opérations avec pipe()**
- Tu peux **gérer les erreurs** avec catchError
- Observable = **flux puissant, flexible, idéal pour les interfaces dynamiques**

CRUD

CRUD = 4 opérations fondamentales

Action Méthode HTTP Description

Create	POST	Ajouter un nouvel élément
Read	GET	Lire (liste ou détail)

Action Méthode HTTP Description

Update PUT ou PATCH Modifier un élément existant

Delete DELETE Supprimer un élément

Laravel

framework PHP open-source, conçu pour rendre le développement web rapide, simple et agréable.

- créer des applications web de manière rapide, propre et sécurisée,
- structurer et organiser le code efficacement,
- bénéficier de nombreuses fonctionnalités prêtes à l'emploi (authentification, base de données, mails, API...).

Fonctionnalité	Description
Artisan	Interface en ligne de commande (CLI) pour générer du code
Eloquent ORM	Manipulation de bases de données via des classes PHP
Routing	Routes web/API simples et puissantes
Blade	Moteur de templates intégré
Migration & Seeder	Gestion versionnée de la base de données
Auth & Middleware	Authentification, permissions, sécurité
Queues, Jobs, Events	Traitement asynchrone, événements système
API Ready	Création facile d'API REST ou Laravel Sanctum/Passport

Commandes de base

Commande	Description
php artisan serve	Lancer le serveur local
php artisan make:model Nom -m	Créer un modèle avec migration

Commande	Description
php artisan migrate	Appliquer les migrations
php artisan make:controller	Créer un contrôleur
php artisan route:list	Afficher toutes les routes